

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15

Total Marks:25

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I Prudhvi Raj.B(192324274) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Prudhvi raj.b

Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. (5 Marks)

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, overparameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability. (5 Marks)

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy. (5 Marks)

7. A reinforcement learning agent fails to learn an optimal policy in a simulated environment. Investigate causes such as poor reward design, insufficient exploration, or incorrect state representation. Explain debugging methods and recommend improvements like reward shaping, exploration strategies, and tuning hyperparameters. (5 Marks)

8. A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis. (5 Marks)

9. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling. (5 Marks)

10. A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies. (5 Marks)

11. A support vector machine (SVM) model trained for classification is producing very low accuracy on both training and test datasets. Analyze possible causes such as improper kernel selection, incorrect hyperparameter tuning (C and gamma), or poor feature scaling. Describe systematic debugging steps to identify the issue and suggest optimization techniques like kernel tuning, normalization, and grid search to improve model performance. (5 Marks)

12. A K-Means clustering algorithm is applied to a dataset, but the clusters formed are highly imbalanced and do not reflect the actual data distribution. Investigate possible issues such as incorrect choice of K, poor initialization of centroids, or presence of outliers. Explain debugging approaches and suggest improvements like the Elbow method, K-Means++, and outlier removal techniques. (5 Marks)

13. A recurrent neural network (RNN) designed for time-series prediction shows inconsistent outputs and fails to capture long-term dependencies. Analyze possible causes such as vanishing gradients, insufficient sequence length, or improper architecture design. Describe debugging steps and propose optimization techniques such as using LSTM/GRU units, gradient clipping, and sequence normalization. (5 Marks)

14. A Naïve Bayes classifier used for text classification performs poorly when applied to real-world data containing noisy and irrelevant features. Examine possible issues such as violation of feature independence assumption, improper preprocessing,

or imbalanced dataset. Explain debugging strategies and suggest improvements like feature selection, smoothing techniques, and data balancing methods. (5 Marks)

15. A reinforcement learning agent trained to solve a navigation problem fails to learn an optimal policy and shows erratic behavior. Analyze possible causes such as sparse rewards, improper exploration-exploitation balance, or incorrect reward function design. Describe debugging approaches and recommend optimization strategies like reward shaping, epsilon decay tuning, and policy optimization methods to improve learning performance. (5 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Question 1

A perceptron on a linearly separable dataset fails to converge even after many iterations – analyze causes and propose debugging/optimization strategies.

Possible Causes

- **Improper learning rate:** A learning rate that is too high causes the weight vector to oscillate past the correct decision boundary instead of settling; a rate that is too low causes updates so tiny that convergence appears not to happen within the given number of iterations.
- **Incorrect weight initialization:** Starting with very large or badly scaled weights can place the initial decision boundary far from the data, requiring an excessive number of corrective updates.
- **Bias handling errors:** If the bias term is omitted, fixed at zero, or not updated along with the weights, the decision boundary is forced to pass through the origin and may never be able to separate the classes correctly even though the data itself is linearly separable.
- **Flawed update rule implementation:** Bugs such as a wrong sign in the update ($w = w - \eta(\text{target} - \text{output})x$ instead of $+$), using the wrong error term, updating only some misclassified points per epoch, or an indexing/off-by-one error in the training loop.

Debugging Strategies

- Re-confirm linear separability by plotting the data (or testing a known separable toy dataset) so the assumption itself is verified before suspecting the algorithm.
- Hand-verify the update rule $w = w + \eta(\text{target} - \text{output})x$ and $b = b + \eta(\text{target} - \text{output})$ on a tiny 2–4 point dataset and compare the code's output against the manual calculation.
- Log the weight vector, bias, and number of misclassifications after every epoch; a constant or randomly jumping misclassification count (rather than a steadily shrinking one) points to a bug in the update logic rather than a genuine optimization difficulty.
- Test the bias update in isolation — run once with bias fixed at 0 and once with bias trainable — to check whether the bias handling is the source of the failure.
- Use the perceptron convergence theorem as a sanity check: for a separable dataset with margin γ and bounded input norm R , convergence is guaranteed within $(R/\gamma)^2$ updates — if far more iterations than this bound have passed with no convergence, the implementation (not the data) is almost certainly at fault.

Optimization Techniques

- Normalize/standardize input features so that all inputs contribute comparably to each update, which increases the effective margin and speeds up convergence.
- Use a moderate learning rate, optionally decayed over epochs, rather than a very large fixed value.
- Shuffle the training examples every epoch to avoid cyclic, repeating update patterns.
- Use the Pocket Algorithm variant (which keeps the best-performing weight vector seen so far) as a safety net while the root cause of non-convergence is being fixed.

Question 2

An MLP trained with backpropagation shows slow convergence and gets stuck in local minima – examine causes and recommend debugging/optimization approaches.

Possible Causes

- **Vanishing gradients:** With sigmoid/tanh activations, neurons saturate for large positive or negative inputs, making their derivatives close to zero; when these small gradients are multiplied backward through many layers, the gradient reaching the early layers becomes negligibly small, so those weights barely update.
- **Inappropriate activation functions:** Using saturating activations (sigmoid/tanh) throughout a deep network, or using a purely linear activation that removes the network's non-linear representational power.
- **Poor weight initialization:** Weights initialized too large push neurons into the saturated region immediately; weights initialized too small make the signal (and its gradient) shrink as it passes through layers.
- **Suboptimal learning rate:** A fixed, poorly chosen learning rate may be too small to escape flat regions/plateaus (a form of shallow local minimum) or too large to settle into a good minimum once found.

Step-by-Step Debugging Approach

- **Step 1:** Plot the training loss curve — a curve that flattens out very early (rather than gradually decreasing) suggests a plateau or local-minimum/vanishing-gradient problem rather than normal slow learning.
- **Step 2:** Inspect the gradient norm layer-by-layer during training; gradients that shrink sharply for the earlier layers compared to the later layers confirm vanishing gradients.
- **Step 3:** Plot histograms of neuron activation outputs per layer — outputs clustered near 0 or 1 (for sigmoid) indicate saturation.
- **Step 4:** Temporarily reduce the network to fewer layers and re-train; if convergence improves significantly, network depth combined with the current activation/initialization is implicated.
- **Step 5:** Swap the activation function (e.g., sigmoid → ReLU) and re-run under identical settings to confirm whether the activation choice was the bottleneck.
- **Step 6:** Run a learning-rate sweep (e.g., 0.001, 0.01, 0.1, 1) and compare loss curves to locate a learning rate that neither stalls nor diverges.

Optimization Techniques

- Replace saturating activations with ReLU/Leaky ReLU/ELU to reduce vanishing gradients.
- Use variance-aware weight initialization matched to the activation function (He initialization for ReLU, Xavier/Glorot initialization for sigmoid/tanh).
- Apply batch normalization to keep layer inputs well-scaled, which stabilizes training and allows larger learning rates.

- Use adaptive optimizers such as Adam or RMSProp, which adjust the effective learning rate per parameter and use momentum to move through shallow local minima and plateaus more effectively than plain gradient descent.
- Apply a learning-rate schedule (decay or cyclical learning rates) so the network can take large steps early on and finer steps as it approaches a good minimum.

Question 3

A genetic algorithm for hypothesis space search gives inconsistent, suboptimal solutions across runs – investigate causes and describe systematic debugging and optimization.

Possible Issues

- **Improper chromosome encoding:** A representation that does not match the problem's structure (e.g., a binary encoding that creates “Hamming cliffs,” where adjacent values differ in many bits) makes crossover and mutation produce poor or invalid offspring.
- **Incorrect fitness function design:** A fitness function that does not accurately reflect the true objective, is poorly scaled (fitness values too close together → weak selection pressure; or one individual's fitness dominating → excessive selection pressure), leads the search astray.
- **Premature convergence:** If selection pressure is too high (e.g., always picking only the very best individuals, or overly aggressive elitism), the population loses diversity early and converges to a local optimum before adequately exploring the search space.
- **Lack of diversity:** A small population size, a very low mutation rate, or repeatedly crossing similar individuals reduces the genetic variety needed to discover better solutions.

Systematic Debugging Steps

- Track a population-diversity metric (e.g., average pairwise Hamming/Euclidean distance, or fitness variance) generation by generation — a sharp early drop signals premature convergence.
- Run several independent trials with different random seeds and compare the final solutions; high variance across runs indicates the algorithm is highly sensitive to initialization/randomness rather than reliably converging.
- Plot best, average, and worst fitness per generation to detect stagnation (flat best-fitness curve while average fitness is still low suggests loss of diversity).
- Hand-test the encoding/decoding and crossover/mutation operators on a small, easily verifiable case to confirm they produce valid, meaningful offspring.
- Validate the fitness function on a few known/benchmark inputs to confirm it ranks candidate solutions in the expected order.

Optimization Strategies

- **Mutation tuning:** Increase the mutation rate if diversity collapses too early; consider an adaptive mutation rate that starts higher (exploration) and decreases over generations (exploitation).
- **Selection pressure adjustment:** Use rank-based or tournament selection with a tunable tournament size instead of raw fitness-proportionate selection, giving finer control over how strongly the fittest individuals are favoured.

- **Elitism, used carefully:** Preserve only a small number (e.g., top 1–2) of the best individuals each generation so good solutions are not lost, without letting elitism dominate and starve diversity.
- Increase population size and periodically introduce new random individuals (“random immigrants”) to sustain diversity throughout the run.
- Redesign the chromosome encoding to better match the problem (e.g., Gray coding to avoid Hamming cliffs, or real-valued encoding for continuous parameters) if encoding issues are confirmed.

Question 4

A DNN combined with genetic programming for a complex prediction problem suffers from overfitting and high computational cost – identify debugging steps and optimization techniques.

Debugging Steps

- Compare training loss/accuracy against validation loss/accuracy over epochs — a widening gap between the two confirms overfitting.
- Compute the ratio of trainable parameters to the number of available training samples; a very high ratio indicates the model is over-parameterized relative to the dataset size.
- Profile training time and resource usage per component (DNN forward/backward pass vs. the GP population-evaluation loop) to locate the actual computational bottleneck rather than assuming it lies in the neural network alone.
- Run an ablation study: shrink the network (fewer layers/neurons) or reduce the GP population/generation count, and check whether validation performance improves and training time drops, confirming over-parameterization and evaluation cost as contributing factors.
- Check whether the amount and diversity of training data is sufficient for the current model capacity (e.g., via a learning curve of validation performance vs. training-set size).

Optimization Techniques

- Apply L1/L2 regularization to penalize large weights and constrain model complexity.
- Use dropout layers during training to reduce neuron co-adaptation and improve generalization.
- Apply model pruning (removing redundant weights/neurons after training, or as part of the GP evolution process) to cut computational cost while retaining most of the accuracy.
- Use efficient/surrogate evaluation during genetic programming (e.g., smaller proxy models, partial training, or fitness estimation) instead of fully training every candidate individual, dramatically reducing evaluation cost per generation.
- Use early stopping based on validation performance to halt training once overfitting begins, and consider data augmentation to effectively enlarge and diversify the training set.

Question 5

A decision tree shows high training accuracy but poor performance on unseen data – analyze causes and propose debugging/optimization methods.

Possible Causes

- **Overfitting due to an unconstrained, deep tree:** The tree keeps splitting until leaves are pure, effectively memorizing the training data (including its noise) instead of learning generalizable rules.
- **Excessively deep tree structure:** Very deep trees with many nodes create highly specific rules that do not generalize to new samples.
- **Noisy or outlier-heavy data:** Noise in the training data causes the tree to create spurious splits that fit random fluctuations rather than the true underlying pattern.

Debugging Steps

- Compare training accuracy with test/validation accuracy — a large gap directly confirms overfitting.
- Visualize the tree's depth and number of leaf nodes relative to the dataset size and number of features to check whether it has grown excessively large.
- Plot a learning curve (accuracy vs. training-set size) to see whether more data would help or whether the issue is purely one of model constraint.
- Examine feature importances to identify irrelevant or noisy features that may be driving unnecessary splits.
- Perform k-fold cross-validation to obtain a more robust estimate of generalization performance and to check for high variance across folds.

Optimization Methods

- **Pruning:** Apply pre-pruning constraints (`max_depth`, `min_samples_split`, `min_samples_leaf`) to limit tree growth, or post-pruning/cost-complexity pruning to simplify an already-grown tree.
- **Cross-validation:** Use k-fold cross-validation to tune hyperparameters such as `max_depth` robustly, instead of relying on a single train/test split.
- **Feature selection:** Remove irrelevant, redundant, or noisy features using importance ranking or statistical tests, reducing the chance that the tree splits on noise.
- Consider ensemble methods such as Random Forest or Gradient Boosting, which combine many trees to reduce variance and typically generalize substantially better than a single deep decision tree.